

```

<script type="text/javascript">

// Pre-load core visualizer components to bypass silent presentation errors
google.charts.load('current', {'packages':['corechart']});
google.charts.setOnLoadCallback(initDashboardExecutionEngine);

let rawCachedDataset = [];

/**
 * Initializes data requests using async server handshakes
 */
function initDashboardExecutionEngine() {
  google.script.run
    .withSuccessHandler(function(processedArrayData) {
      rawCachedDataset = processedArrayData;

      // Update presentation displays
      document.getElementById('loadingSpinner').style.display = 'none';
      document.getElementById('dashboardContent').style.display = 'block';

      populateDynamicFilters();
      updateDashboard();

      // Global structural resizing listener for true chart responsivity
      window.addEventListener('resize', updateDashboard);
    })
    .withFailureHandler(function(errorObj) {
      document.getElementById('loadingSpinner').innerHTML =
        '<div class="container py-5"><div class="alert alert-danger shadow-lg border-0 p-4">' +
        '<h4 class="fw-bold text-danger">Data Stream Processing Execution Fault</h4><hr>' +
        '<p class="mb-0 font-monospace text-dark">' + errorObj.message + '</p></div></div>';
    })
}

```

```

    .getDashboardData();
}

/**
 * Dynamically populates control options from the actual contents of the sheet
 */
function populateDynamicFilters() {
    const uniqueRegions = new Set();
    const uniqueCategories = new Set();

    rawCachedDataset.forEach(transaction => {
        if (transaction.region) uniqueRegions.add(transaction.region);
        if (transaction.category) uniqueCategories.add(transaction.category);
    });

    const regionDropdown = document.getElementById('filterRegion');
    const categoryDropdown = document.getElementById('filterCategory');

    Array.from(uniqueRegions).sort().forEach(reg => {
        let optionNode = document.createElement('option');
        optionNode.value = reg;
        optionNode.innerText = reg;
        regionDropdown.appendChild(optionNode);
    });

    Array.from(uniqueCategories).sort().forEach(cat => {
        let optionNode = document.createElement('option');
        optionNode.value = cat;
        optionNode.innerText = cat;
        categoryDropdown.appendChild(optionNode);
    });
}

```

```

}

/**
 * Core execution pipeline: applies filters, computes metrics, and re-renders graphs
 */
function updateDashboard() {
  const activeRegion = document.getElementById('filterRegion').value;
  const activeCategory = document.getElementById('filterCategory').value;

  // Evaluate operational data array based on active filters
  const processedSubset = rawCachedDataset.filter(row => {
    const regionValidation = (activeRegion === 'All' || row.region === activeRegion);
    const categoryValidation = (activeCategory === 'All' || row.category === activeCategory);
    return regionValidation && categoryValidation;
  });

  // Handle edge-case configurations with empty returns gracefully
  if (processedSubset.length === 0) {
    document.getElementById('kpiRevenue').innerText = "$0.00";
    document.getElementById('kpiUnits').innerText = "0";
    document.getElementById('kpiRating').innerText = "0.00 / 5.00";
    document.getElementById('kpiDiscount').innerText = "0.0%";

    const fallbackMsg = '<div class="no-data-element">No active metrics match current selection filter values.</div>';

    document.getElementById('trendChart').innerHTML = fallbackMsg;
    document.getElementById('categoryChart').innerHTML = fallbackMsg;
    document.getElementById('regionChart').innerHTML = fallbackMsg;
    document.getElementById('memberChart').innerHTML = fallbackMsg;

    return;
  }

```

```

// Compute KPI Aggregations

let summaryRevenue = 0;

let summaryUnits = 0;

let accumulatedRatings = 0;

let accumulatedDiscount = 0;


processedSubset.forEach(row => {

    summaryRevenue += row.totalSales;

    summaryUnits += row.quantity;

    accumulatedRatings += row.rating;

    accumulatedDiscount += row.discount;

});


const calculatedMeanRating = accumulatedRatings / processedSubset.length;

const calculatedMeanDiscount = accumulatedDiscount / processedSubset.length;


// Write Formatted String Data outputs to KPIs

document.getElementById('kpiRevenue').innerText = new Intl.NumberFormat('en-US', { style:
'currency', currency: 'USD' }).format(summaryRevenue);

document.getElementById('kpiUnits').innerText = new Intl.NumberFormat('en-
US').format(summaryUnits);

document.getElementById('kpiRating').innerText = calculatedMeanRating.toFixed(2) + " /
5.00";

document.getElementById('kpiDiscount').innerText = (calculatedMeanDiscount *
100).toFixed(1) + "%";


// --- CHART GRAPH AGGREGATIONS ---


// Chart 1: Monthly Line Velocity Data Setup

const operationalMonthNames = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov',
'Dec'];

let monthlyRevenueMap = Array(12).fill(0);

```

```

processedSubset.forEach(row => {
  if (row.month >= 0 && row.month < 12) {
    monthlyRevenueMap[row.month] += row.totalSales;
  }
});

let trendTable = new google.visualization.DataTable();
trendTable.addColumn('string', 'Month');
trendTable.addColumn('number', 'Revenue');
for(let i = 0; i < 12; i++) {
  trendTable.addRow([operationalMonthNames[i], monthlyRevenueMap[i]]);
}

// Chart 2: Category Bar Breakdown (Sorted Descending)
let categoryAggregationMap = {};
processedSubset.forEach(row => {
  categoryAggregationMap[row.category] = (categoryAggregationMap[row.category] || 0) +
row.totalSales;
});

let categoryTable = new google.visualization.DataTable();
categoryTable.addColumn('string', 'Product Category');
categoryTable.addColumn('number', 'Revenue');
Object.keys(categoryAggregationMap)
.sort((a, b) => categoryAggregationMap[b] - categoryAggregationMap[a])
.forEach(catKey => {
  categoryTable.addRow([catKey, categoryAggregationMap[catKey]]);
});

// Chart 3: Regional Sales Share Pie Data Setup
let regionalAggregationMap = {};
processedSubset.forEach(row => {

```

```

    regionalAggregationMap[row.region] = (regionalAggregationMap[row.region] || 0) +
row.totalSales;

});

let regionTable = new google.visualization.DataTable();

regionTable.addColumn('string', 'Territory Region');

regionTable.addColumn('number', 'Revenue');

Object.keys(regionalAggregationMap).sort().forEach(regKey => {
    regionTable.addRow([regKey, regionalAggregationMap[regKey]]);
});

// Chart 4: Membership Segment Distribution Data Setup

let subscriberAggregationMap = { 'Loyalty Program Member': 0, 'Regular Walk-In Guest': 0 };

processedSubset.forEach(row => {
    if (row.isMember) {
        subscriberAggregationMap['Loyalty Program Member'] += row.totalSales;
    } else {
        subscriberAggregationMap['Regular Walk-In Guest'] += row.totalSales;
    }
});

let membershipTable = new google.visualization.DataTable();

membershipTable.addColumn('string', 'Customer Profile Class');

membershipTable.addColumn('number', 'Revenue');

membershipTable.addRow(['Loyalty Program Member', subscriberAggregationMap['Loyalty
Program Member']]);

membershipTable.addRow(['Regular Walk-In Guest', subscriberAggregationMap['Regular
Walk-In Guest']]);

// --- CONFIGURE STYLING OBJECT PROPERTIES FOR CHART AREAS ---

const corporatePaletteColors = ['#3b82f6', '#10b981', '#06b6d4', '#f59e0b', '#8b5cf6',
'#ec4899'];

let lineOptions = {

```

```

title: 'Monthly Sales Revenue Velocity (Fiscal Year 2023)',
titleTextStyle: { fontSize: 14, color: '#1e293b', bold: true },
curveType: 'function',
legend: { position: 'none' },
colors: [corporatePaletteColors[0]],
lineWidth: 4,
chartArea: { width: '85%', height: '70%', top: 50 },
vAxis: { format: '$#,###', gridlines: { color: '#f1f5f9' }, textStyle: { color: '#64748b', fontSize: 11 }
},
hAxis: { textStyle: { color: '#64748b', fontSize: 11 } }
};

```

```

let barOptions = {
title: 'Total Revenue Contribution by Department Category',
titleTextStyle: { fontSize: 14, color: '#1e293b', bold: true },
legend: { position: 'none' },
colors: [corporatePaletteColors[1]],
chartArea: { width: '85%', height: '70%', top: 50 },
vAxis: { format: '$#,###', gridlines: { color: '#f1f5f9' }, textStyle: { color: '#64748b', fontSize: 11 }
},
hAxis: { textStyle: { color: '#64748b', fontSize: 11 } }
};

```

```

let donutRegionOptions = {
title: 'Geographic Sales Contribution Split',
titleTextStyle: { fontSize: 14, color: '#1e293b', bold: true },
pieHole: 0.45,
colors: corporatePaletteColors,
chartArea: { width: '90%', height: '75%', top: 50 },
legend: { position: 'right', textStyle: { color: '#334155', fontSize: 11 } }
};

```

```

let donutMemberOptions = {
  title: 'Customer Loyalty Revenue Value Share',
  titleTextStyle: { fontSize: 14, color: '#1e293b', bold: true },
  pieHole: 0.45,
  colors: ['#6366f1', '#f43f5e'],
  chartArea: { width: '90%', height: '75%', top: 50 },
  legend: { position: 'right', textStyle: { color: '#334155', fontSize: 11 } }
};

// Draw and bind elements to the UI safely without overlapping bugs

new
google.visualization.LineChart(document.getElementById('trendChart')).draw(trendTable,
lineOptions);

new
google.visualization.ColumnChart(document.getElementById('categoryChart')).draw(catTable,
barOptions);

new
google.visualization.PieChart(document.getElementById('regionChart')).draw(regionTable,
donutRegionOptions);

new
google.visualization.PieChart(document.getElementById('memberChart')).draw(membershipTa
ble, donutMemberOptions);
}
</script>

```